

BÀI 3: BẢO MẬT MẠNG MÁY TÍNH

Bài học này trang bị kiến thức về lập trình socket an toàn với SSL/TLS và xác minh chứng chỉ. Sinh viên sẽ thực hành xây dựng ứng dụng chat bảo mật với mã hóa đầu-cuối, xác thực client, quản lý kết nối và phòng chat. Đồng thời, sinh viên sẽ phát triển công cụ trình sát mạng để quét cổng, nhận dạng dịch vụ và kiểm tra lỗ hổng, qua đó nắm vững kỹ thuật bảo mật và phân tích hệ thống mạng.

3.1 LẬP TRÌNH SOCKET AN TOÀN

3.1.1 Lập trình socket bảo mật

Socket là một điểm cuối (endpoint) cho giao tiếp hai chiều giữa hai tiến trình qua mạng. Trong lập trình mạng, socket là giao diện lập trình (API) cho phép các chương trình thiết lập kết nối, gửi và nhận dữ liệu qua giao thức TCP hoặc UDP. Socket thông thường không cung cấp tính bảo mật, dữ liệu truyền qua có thể bị nghe lén (eavesdropping), giả mạo (spoofing), hoặc bị sửa đổi trong quá trình truyền.

Để đảm bảo an toàn cho dữ liệu, lập trình socket cần tích hợp các cơ chế mã hóa và xác thực. Giải pháp phổ biến là sử dụng SSL/TLS để tạo socket bảo mật (secure socket), cho phép:

- Mã hóa toàn bộ dữ liệu truyền đi.
- Xác minh danh tính hai bên (thông qua chứng chỉ số).
- Phòng chống tấn công nghe lén và xen giữa (MITM).

Trong môi trường bảo mật, server phải cấu hình SSL/TLS và cung cấp chứng chỉ số tin cậy. Client cần xác thực chứng chỉ và thiết lập kênh truyền mã hóa trước khi trao đổi dữ liệu.

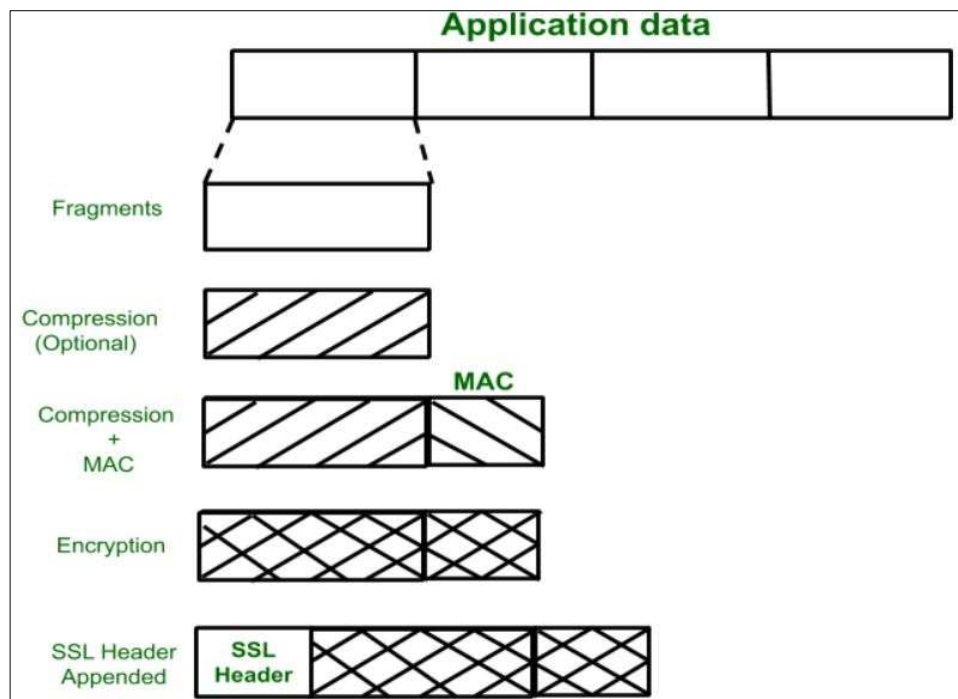
3.1.2 SSL/TLS

SSL (Secure Sockets Layer) và **TLS (Transport Layer Security)** là các giao thức mật mã dùng để bảo vệ dữ liệu khi truyền qua mạng. TLS là phiên bản kế thừa, an toàn hơn và đang được sử dụng phổ biến hơn. Nguyên lý hoạt động:

- Bắt tay TLS (TLS Handshake): Hai bên trao đổi thông tin để thiết lập khóa phiên (session key), sử dụng mã hóa bất đối xứng.
- Mã hóa phiên: Sau khi bắt tay, dữ liệu được mã hóa bằng thuật toán đối xứng để tăng hiệu suất.
- Xác thực: Server cung cấp chứng chỉ số để client xác minh danh tính, đảm bảo không bị giả mạo.

Thành phần chính của TLS:

- Chứng chỉ số (Certificate): Xác thực danh tính server (hoặc client) qua bên thứ ba đáng tin cậy (CA).
- Cipher suite: Tập hợp các thuật toán dùng trong quá trình mã hóa, bắt tay và kiểm tra toàn vẹn dữ liệu.

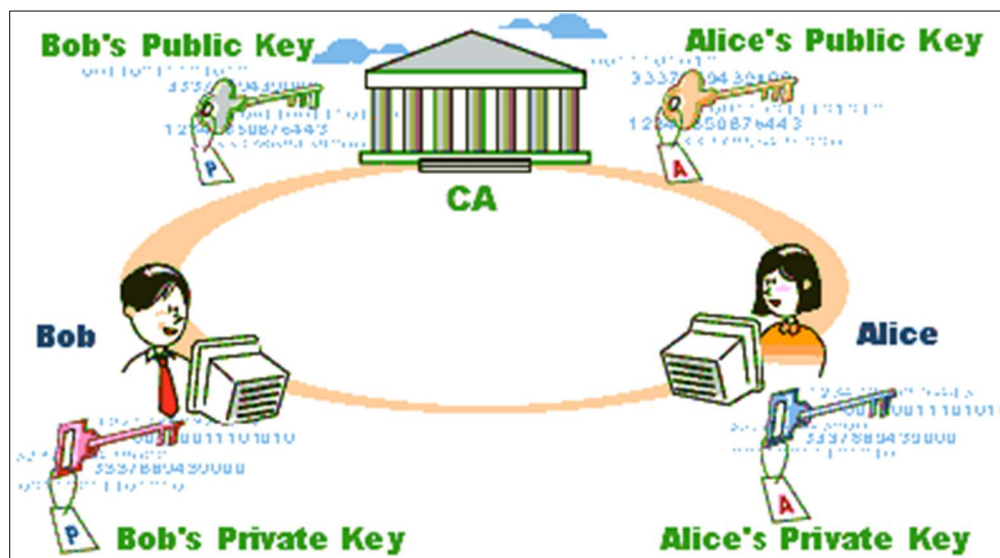


Hình 3.1. Secure Socket Layer. Nguồn: GeeksforGeeks

3.1.3 Xác minh chứng chỉ

Chứng chỉ số là tài liệu điện tử do một bên thứ ba đáng tin cậy (**Certificate Authority – CA**) phát hành, chứa thông tin về danh tính, khóa công khai và thời hạn hiệu lực. Quy trình xác minh chứng chỉ:

- Dùng khóa công khai của CA để kiểm tra chữ ký trên chứng chỉ, đảm bảo chứng chỉ chưa bị thay đổi hoặc giả mạo.
- Xác minh lần lượt từ chứng chỉ hiện tại lên đến chứng chỉ gốc (Root CA) được tin cậy.
- Đảm bảo chứng chỉ chưa hết hạn hoặc chưa được sử dụng trước ngày bắt đầu hiệu lực.
- Xác nhận chứng chỉ chưa bị thu hồi do bị lộ khóa hay vi phạm.
- So sánh thông tin trong chứng chỉ với đối tượng kết nối để tránh giả mạo.



Hình 3.2. Vai trò của CA trong PKI. Nguồn: Savvy Security

3.2 THỰC HÀNH: SECURECHAT

3.2.1 Yêu cầu

Xây dựng ứng dụng chat bảo mật gồm:

- SecureChatServer: Server đa luồng hỗ trợ SSL/TLS.
- SecureChatClient: Client xác minh chứng chỉ.
- MessageEncryption: Mã hóa đầu-cuối cho tin nhắn.
- ConnectionManager: Quản lý kết nối client an toàn.

- RoomManager: Hỗ trợ nhiều phòng chat.

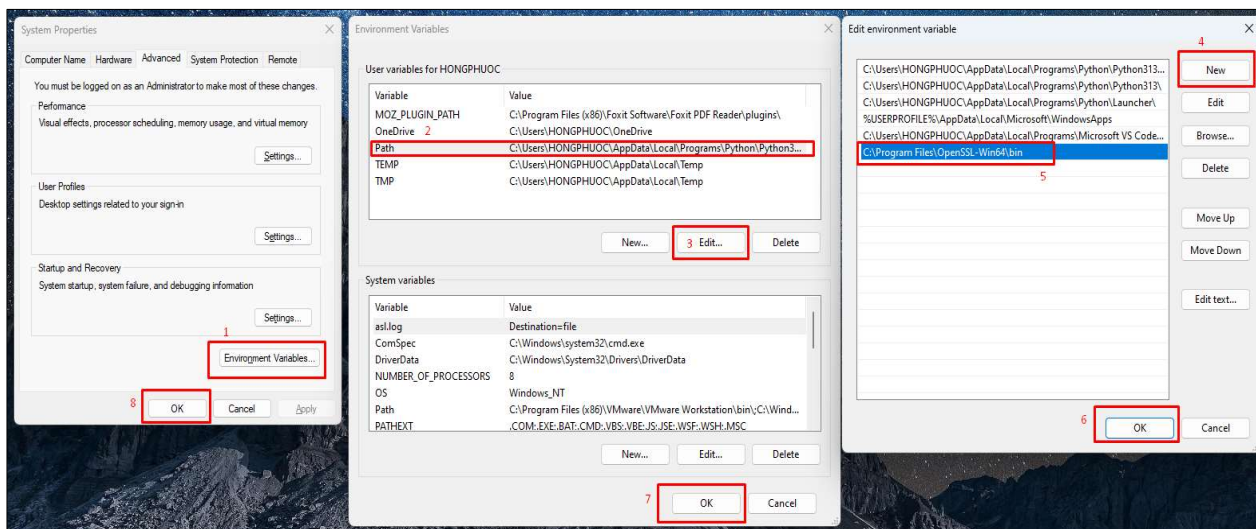
3.2.2 Hướng dẫn thực hành

- Vào trang <https://slproweb.com/products/Win32OpenSSL.html>> Chọn phiên bản phù hợp với hệ điều hành và tải về.

Download Win32/Win64 OpenSSL today using the links below!

File	Type	Description
Win64 OpenSSL v3.5.0 Light EXE MSI	5MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.5.0 (Recommended for users by chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More in
Win64 OpenSSL v3.5.0 EXE MSI	280MB Installer	Installs Win64 OpenSSL v3.5.0 (Recommended for software developers by the creators of OpenSSL default build of OpenSSL and is subject to local and state laws. More information can be found in t
Win32 OpenSSL v3.5.0 Light EXE MSI	4MB Installer	Installs the most commonly used essentials of Win32 OpenSSL v3.5.0 (Only install this if you need and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v3.5.0 EXE MSI	232MB Installer	Installs Win32 OpenSSL v3.5.0 (Only install this if you need 32-bit OpenSSL for Windows). Note t

- Tiến hành cài đặt như các phần mềm thông thường.
- Thêm "openssl" vào PATH của Windows:



- Kiểm tra trong Terminal:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> openssl
help:

Standard commands
asn1parse      ca              ciphers         cmp
cms             crl             crl2pkcs7      dgst
dhparam        dsa             dsaparam       ec
ecparam        enc             engine          errstr
fipsinstall    gendsa         genpkey         genrsa
help           info           kdf             list

```

- Tạo thư mục "secure-chat". Trong thư mục này tạo thư mục "certs".
- Trong thư mục "secure-chat", tạo file "openssl.cnf":

secure-chat > openssl.cnf

```
1 [req]
2 distinguished_name = req_distinguished_name
3 x509_extensions = v3_ca
4 prompt = no
5
6 [req_distinguished_name]
7 C = VN
8 ST = HN
9 L = HN
10 O = MyOrg
11 OU = IT Dept
12 CN = MyRootCA
13
14 [v3_ca]
15 subjectKeyIdentifier = hash
16 authorityKeyIdentifier = keyid:always,issuer
17 basicConstraints = critical, CA:true
18 keyUsage = critical, keyCertSign, cRLSign
19
```

- Tạo file "make-certs.bat" có nội dung như sau:

```
@echo off

setlocal enabledelayedexpansion

:: Di chuyển vào thư mục hiện tại
cd /d %~dp0

:: Tạo các thư mục con trong certs/
mkdir certs\ca
mkdir certs\server
mkdir certs\client

:: CA
openssl genrsa -out certs\ca\ca.key 2048
openssl req -x509 -new -nodes -key certs\ca\ca.key -sha256 -days 3650 -
out certs\ca\ca.crt -config openssl.cnf -extensions v3_ca

:: Server
openssl genrsa -out certs\server\server.key 2048
```

```
openssl req -new -key certs\server\server.key -out
certs\server\server.csr -subj "/C=VN/ST=HN/L=HN/O=MyOrg/OU=IT
Dept/CN=localhost"

openssl x509 -req -in certs\server\server.csr -CA certs\ca\ca.crt -CAkey
certs\ca\ca.key -CAcreateserial -out certs\server\server.crt -days 365 -
sha256

:: Client

openssl genrsa -out certs\client\client.key 2048

openssl req -new -key certs\client\client.key -out
certs\client\client.csr -subj "/C=VN/ST=HN/L=HN/O=MyOrg/OU=IT
Dept/CN=client"

openssl x509 -req -in certs\client\client.csr -CA certs\ca\ca.crt -CAkey
certs\ca\ca.key -CAcreateserial -out certs\client\client.crt -days 365 -
sha256

:: Đổi tên file serial để tránh ðề

move certs\ca\ca.srl certs\ca\ca.srl.bak >nul 2>&1

echo.

echo =====

echo Cac chung chi da tao xong!

echo - CA:      certs\ca\

echo - Server:  certs\server\

echo - Client:  certs\client\

echo =====

pause
```

```

secure-chat > make-certs.bat
1 @echo off
2 setlocal enabledelayedexpansion
3
4 :: Di chuyển vào thư mục hiện tại
5 cd /d %~dp0
6
7 :: Tạo các thư mục con trong certs/
8 mkdir certs\ca
9 mkdir certs\server
10 mkdir certs\client
11
12 :: CA
13 openssl genrsa -out certs\ca\ca.key 2048
14 openssl req -x509 -new -nodes -key certs\ca\ca.key -sha256 -days 3650 -out certs\ca\ca.crt
   -extensions v3_ca
15
16 :: Server

```

- Chạy file "make-certs.bat":

```

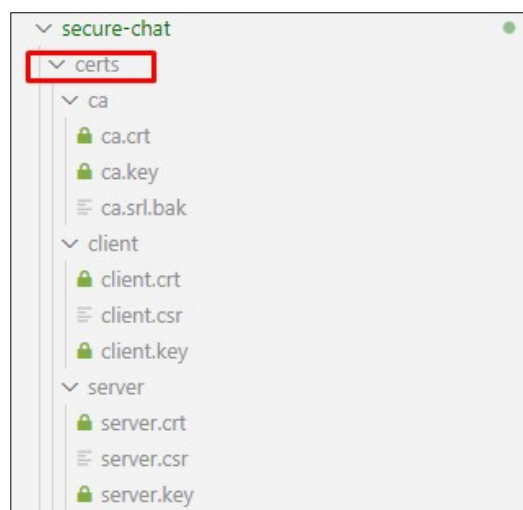
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat> .\make-certs.bat
Certificate request self-signature ok
subject=C=VN, ST=HN, L=HN, O=MyOrg, OU=IT Dept, CN=localhost
Certificate request self-signature ok
subject=C=VN, ST=HN, L=HN, O=MyOrg, OU=IT Dept, CN=client

=====
Cac chung chi da tao xong
- CA:      certs\ca\
- Server:  certs\server\
- Client:  certs\client\
=====
Press any key to continue . . .
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat>

```

- Kiểm tra trong thư mục "certs", thấy có các thư mục và chứng chỉ đã được tạo:



- Code cho file "message_encryption.py":

```

1 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
2 from cryptography.hazmat.primitives import padding
3 from cryptography.hazmat.backends import default_backend
4 import os
5
6 class MessageEncryption:
7     def __init__(self, key=None):
8         self.key = key or os.urandom(32) # 256-bit AES key
9         self.backend = default_backend()
10
11     def encrypt(self, plaintext):
12         iv = os.urandom(16)
13         cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv), backend=self.backend)
14         encryptor = cipher.encryptor()
15
16         padder = padding.PKCS7(128).padder()
17         padded_data = padder.update(plaintext.encode('utf-8')) + padder.finalize()
18
19         ct = encryptor.update(padded_data) + encryptor.finalize()
20         return iv + ct
21
22     def decrypt(self, ciphertext):
23         iv = ciphertext[:16]
24         ct = ciphertext[16:]
25         cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv), backend=self.backend)
26         decryptor = cipher.decryptor()
27         padded_data = decryptor.update(ct) + decryptor.finalize()
28
29         unpadder = padding.PKCS7(128).unpadder()
30         data = unpadder.update(padded_data) + unpadder.finalize()
31         return data.decode('utf-8')
32

```

secure-chat > connection_manager.py > ConnectionManager > add_client

[illegible]

```

13     def remove_client(self, client_sock):
14         with self.lock:
15             if client_sock in self.clients:
16                 del self.clients[client_sock]
17
18     def get_client(self, client_sock):
19         with self.lock:
20             return self.clients.get(client_sock)
21
22     def broadcast(self, message, sender_sock):
23         with self.lock:
24             for client in self.clients:
25                 if client != sender_sock:
26                     try:
27                         client.send(message)
28                     except Exception:
29                         pass
30

```

- Code cho file "room_manager.py":

```

1  import threading
2
3  class RoomManager:
4      def __init__(self):
5          self.rooms = {} # room_name -> set(client sockets)
6          self.lock = threading.Lock()
7
8      def create_room(self, room_name):
9          with self.lock:
10             if room_name not in self.rooms:
11                 self.rooms[room_name] = set()
12
13      def join_room(self, room_name, client_sock):
14          with self.lock:
15             if room_name not in self.rooms:
16                 self.rooms[room_name] = set()
17             self.rooms[room_name].add(client_sock)
18
19      def leave_room(self, room_name, client_sock):
20          with self.lock:
21             if room_name in self.rooms:
22                 self.rooms[room_name].discard(client_sock)
23

```

```

24     def broadcast_room(self, room_name, message, sender_sock):
25         with self.lock:
26             if room_name not in self.rooms:
27                 return
28             for client in self.rooms[room_name]:
29                 if client != sender_sock:
30                     try:
31                         client.send(message)
32                     except Exception:
33                         pass
34

```

- Code cho file "server.py":

```

secure-chat > server.py > ...
1  import socket
2  import ssl
3  import threading
4  from connection_manager import ConnectionManager
5  from room_manager import RoomManager
6  from message_encryption import MessageEncryption
7
8  HOST = '127.0.0.1'
9  PORT = 8443
10
11  SERVER_CERT = 'certs/server/server.crt'
12  SERVER_KEY = 'certs/server/server.key'
13  CA_CERT = 'certs/ca/ca.crt'
14
15  connection_manager = ConnectionManager()
16  room_manager = RoomManager()
17
18  def handle_client(connstream, addr):
19      print(f"[+] Client connected: {addr}")
20      try:
21          # Bước 1: nhận username và AES key
22          # Đơn giản giả định client gửi: "username:key"
23          data = connstream.recv(1024).decode()
24          if ':' not in data:
25              connstream.close()
26              return
27          username, key_hex = data.split(':')
28          encryption_key = bytes.fromhex(key_hex)
29
30          connection_manager.add_client(connstream, username, encryption_key)
31

```

```

32     # Mặc định join phòng "general"
33     room_manager.create_room('general')
34     room_manager.join_room('general', connstream)
35
36     me = MessageEncryption(encryption_key)
37
38     while True:
39         enc_message = connstream.recv(4096)
40         if not enc_message:
41             break
42         try:
43             message = me.decrypt(enc_message)
44         except Exception:
45             print("[!] Decryption failed")
46             continue
47
48         print(f"[{username}]: {message}")
49
50         # Mã hóa lại message gửi cho phòng, kèm tên user
51         out_msg = f"[{username}]: {message}"
52         # Mã hóa từng client khác theo key của họ
53         with connection_manager.lock:
54             for client_sock, info in connection_manager.clients.items():
55                 if client_sock != connstream:
56                     try:
57                         me_other = MessageEncryption(info['encryption_key'])
58                         enc_out = me_other.encrypt(out_msg)
59                         client_sock.send(enc_out)
60                     except Exception:
61                         pass
62

```

```

63     except Exception as e:
64         print(f"Exception {e}")
65     finally:
66         print(f"[-] Client disconnected: {addr}")
67         connection_manager.remove_client(connstream)
68         room_manager.leave_room('general', connstream)
69         try:
70             connstream.shutdown(socket.SHUT_RDWR)
71         except Exception:
72             pass
73         connstream.close()
74

```

```

75 def main():
76     context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
77     context.load_cert_chain(certfile=SERVER_CERT, keyfile=SERVER_KEY)
78     context.load_verify_locations(cafile=CA_CERT)
79     context.verify_mode = ssl.CERT_REQUIRED # Yêu cầu client chứng chỉ
80     context.options |= ssl.OP_NO_TLSv1 | ssl.OP_NO_TLSv1_1 # TLS1.2 trở lên
81
82     bindsocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
83     bindsocket.bind((HOST, PORT))
84     bindsocket.listen(5)
85     print(f"Server listening on {HOST}:{PORT}")
86
87     while True:
88         newsocket, fromaddr = bindsocket.accept()
89         try:
90             connstream = context.wrap_socket(newsocket, server_side=True)
91             threading.Thread(target=handle_client,
92                             args=(connstream, fromaddr), daemon=True).start()
93         except ssl.SSLError as e:
94             print(f"SSL Error: {e}")
95
96 if __name__ == '__main__':
97     main()
98

```

- Code cho file "client.py":

```

secure-chat > client.py > ...
1  import socket
2  import ssl
3  import threading
4  import os
5  import binascii
6  from message_encryption import MessageEncryption
7
8  SERVER_HOST = '127.0.0.1'
9  SERVER_PORT = 8443
10
11 CA_CERT = 'certs/ca/ca.crt'
12 CLIENT_CERT = 'certs/client/client.crt'
13 CLIENT_KEY = 'certs/client/client.key'
14

```

```

15 def receive_messages(ssl_sock, me):
16     try:
17         while True:
18             enc_data = ssl_sock.recv(4096)
19             if not enc_data:
20                 break
21             try:
22                 msg = me.decrypt(enc_data)
23                 print(msg)
24             except Exception:
25                 print("[!] Failed to decrypt message")
26     except Exception:
27         pass
28
29 def main():
30     username = input("Username: ").strip()
31     # Khởi tạo key AES 256
32     aes_key = os.urandom(32)
33     me = MessageEncryption(aes_key)
34
35     context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH,
36                                         cafile=CA_CERT)
37     context.load_cert_chain(certfile=CLIENT_CERT, keyfile=CLIENT_KEY)
38     context.check_hostname = False
39     context.verify_mode = ssl.CERT_REQUIRED
40
41     sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
42     ssl_sock = context.wrap_socket(sock, server_hostname=SERVER_HOST)
43     ssl_sock.connect((SERVER_HOST, SERVER_PORT))
44
45     # Gửi username + key hex cho server
46     ssl_sock.send(f"{username}:{binascii.hexlify(aes_key).decode()}".encode())
47

```

```

48     threading.Thread(target=receive_messages,
49                     args=(ssl_sock, me), daemon=True).start()
50
51     print("Type messages (type 'exit' to quit):")
52     while True:
53         msg = input()
54         if msg.lower() == 'exit':
55             break
56         enc_msg = me.encrypt(msg)
57         ssl_sock.send(enc_msg)
58
59     ssl_sock.close()
60
61 if __name__ == '__main__':
62     main()
63

```

- Chạy "server.py" và "client.py" để kiểm tra:



```
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat> python .\server.py
C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat\server.py:80: DeprecationWarning: ssl
1.OP_NO_SSL*/ssl.OP_NO_TLS* options are deprecated
context.options |= ssl.OP_NO_TLSv1 | ssl.OP_NO_TLSv1_1 # TLS1.2 trở lên
Server listening on 127.0.0.1:8443
[+] Client connected: ('127.0.0.1', 58419)
[phuc]: xin chào

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat> python .\client.py
Username: phuc
Type messages (type 'exit' to quit):
xin chào
```

- Chạy thêm 1 client để kiểm tra:

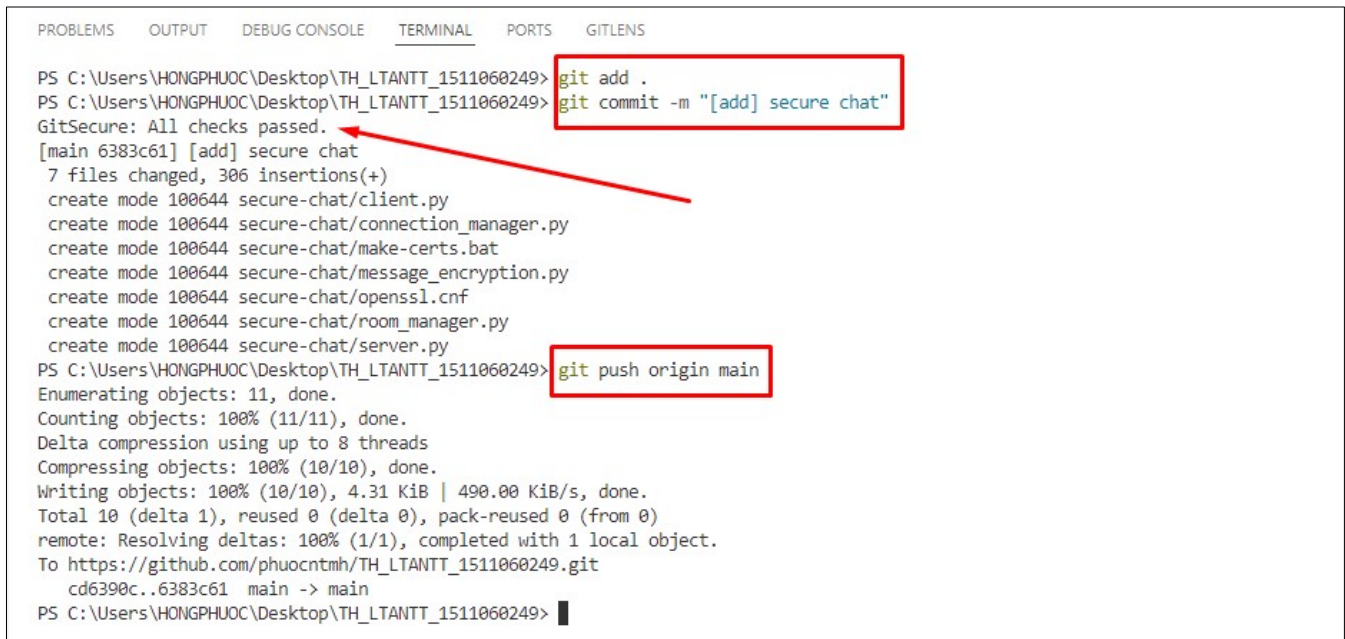


```
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat> python .\server.py
C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat\server.py:80: DeprecationWarning: ssl
1.OP_NO_SSL*/ssl.OP_NO_TLS* options are deprecated
context.options |= ssl.OP_NO_TLSv1 | ssl.OP_NO_TLSv1_1 # TLS1.2 trở lên
Server listening on 127.0.0.1:8443
[+] Client connected: ('127.0.0.1', 58431)
[phuc]: xin chào
[+] Client connected: ('127.0.0.1', 58432)
[ty]: hello
[phuc]: day la ung dung chat ma hoa
[ty]: duoc ma hoa bang ssl

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> cd .\secure-chat\
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat> python .\client.py
Username: phuc
Type messages (type 'exit' to quit):
xin chào
[ty]: hello
[phuc]: day la ung dung chat ma hoa
[ty]: duoc ma hoa bang ssl

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> cd .\secure-chat\
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\secure-chat> python .\client.py
Username: ty
Type messages (type 'exit' to quit):
hello
[phuc]: day la ung dung chat ma hoa
duoc ma hoa bang ssl
```

- Commit code, push code lên Github:



```
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git add .
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git commit -m "[add] secure chat"
GitSecure: All checks passed.
[main 6383c61] [add] secure chat
7 files changed, 306 insertions(+)
create mode 100644 secure-chat/client.py
create mode 100644 secure-chat/connection_manager.py
create mode 100644 secure-chat/make-certs.bat
create mode 100644 secure-chat/message_encryption.py
create mode 100644 secure-chat/openssl.cnf
create mode 100644 secure-chat/room_manager.py
create mode 100644 secure-chat/server.py
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 4.31 KiB | 490.00 KiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/phuocntmh/TH_LTANTT_1511060249.git
cd6390c..6383c61 main -> main
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249>
```

3.3 QUÉT MẠNG VÀ TRÌNH SÁT

3.3.1 Kỹ thuật quét cổng

Quét cổng là quá trình gửi các gói tin đến các cổng mạng (ports) trên máy chủ hoặc thiết bị để xác định trạng thái của chúng (mở, đóng hoặc được lọc).

Các trạng thái cổng phổ biến:

- Open: Cổng đang mở và có dịch vụ đang lắng nghe.
- Closed: Cổng không có dịch vụ, nhưng thiết bị phản hồi.

- Filtered: Gói tin bị chặn, không có phản hồi rõ ràng (thường do tường lửa).
- Kỹ thuật quét cổng phổ biến:
- TCP Connect Scan: Dùng `connect()` để thiết lập kết nối đầy đủ; dễ phát hiện nhưng đơn giản.
- TCP SYN Scan: Gửi gói SYN và quan sát phản hồi, khó bị phát hiện hơn.
- UDP Scan: Gửi gói UDP tới cổng; khó xác định hơn vì không phản hồi rõ ràng.
- Xmas Scan, FIN Scan, NULL Scan: Gửi gói TCP bất thường để dò phản ứng hệ thống (phục vụ bypass firewall).
- Idle Scan: Tận dụng máy bên thứ ba để quét ẩn danh (ẩn địa chỉ IP thật).

3.3.2 Nhận dạng dịch vụ

Service Fingerprinting là quá trình phân tích phản hồi từ các cổng dịch vụ để suy ra thông tin về ứng dụng hoặc phần mềm phía sau các cổng đó – ví dụ: Apache 2.4.51, OpenSSH 8.9p1, ... Phân loại:

- Passive Fingerprinting: Phân tích lưu lượng mà không tương tác trực tiếp với dịch vụ.
- Active Fingerprinting: Gửi truy vấn có chủ đích và phân tích phản hồi để xác định dấu hiệu đặc trưng.

Kỹ thuật thường dùng:

- So sánh định dạng banner phản hồi (banner matching).
- Phân tích chi tiết giao thức phản hồi (ví dụ: HTTP headers, SMTP greeting).
- Dựa vào hành vi phản hồi bất thường với gói tin không hợp lệ.
- Kết hợp nhiều tiêu chí để đoán phiên bản dịch vụ chính xác hơn.

3.4 THỰC HÀNH: NETRECON

3.4.1 Yêu cầu

Xây dựng bộ công cụ khám phá mạng Netrecon gồm:

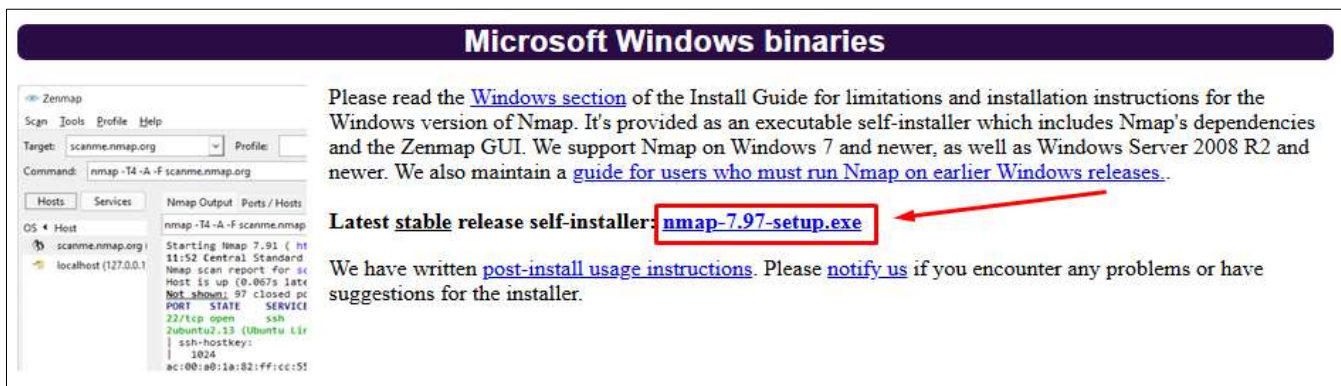
- `PortScanner`: Quét cổng TCP/UDP với kỹ thuật ẩn mình.

- ServiceDetector: Phát hiện phiên bản dịch vụ.
- BannerGrabber: Lấy banner an toàn.
- NetworkMapper: Khám phá sơ đồ mạng.
- VulnChecker: Phát hiện lỗ hổng cơ bản.

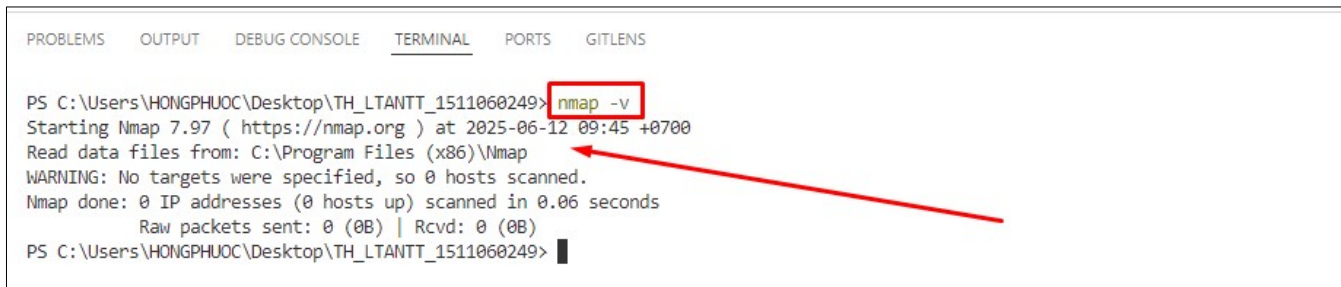
Công cụ phải tuân thủ giới hạn tốc độ (rate limiting), ghi log tất cả hoạt động với thời gian, hỗ trợ whitelist/blacklist.

3.4.2 Hướng dẫn thực hành

- Tải Nmap tại trang <https://nmap.org/download.html>



- Tiến hành cài đặt vào máy tính.
- Kiểm tra xem Nmap đã được cài đặt hay chưa trong Terminal:



- Nếu Nmap chưa nhận, hãy thêm Nmap vào PATH của Windows, khởi động lại VS Code và thử lại.
- Lấy STMP_PASS bằng cách truy cập <https://myaccount.google.com/apppasswords>
- Tạo app mới > Đặt tên là "Netrecom" > Lưu giữ STMP_PASS.

← Mật khẩu ứng dụng

Mật khẩu ứng dụng giúp bạn đăng nhập vào Tài khoản Google của mình trên những ứng dụng và dịch vụ cũ không hỗ trợ các tiêu chuẩn bảo mật hiện đại.

Mật khẩu ứng dụng kém an toàn hơn so với việc sử dụng những ứng dụng và dịch vụ mới nhất hỗ trợ các tiêu chuẩn bảo mật hiện đại. Trước khi tạo mật khẩu ứng dụng, bạn nên kiểm tra xem ứng dụng của mình có cần mật khẩu này để đăng nhập hay không.

[Tìm hiểu thêm](#)

Mật khẩu ứng dụng của bạn

Netrecon

Ngày tạo: 08:46



- Tạo folder "netrecon" trong folder source code clone về từ Github.
- Trong folder "netrecon" tạo các folder "modules", "static", "templates" và các file "requirements.txt", "cli.py", "app.py", ".env".
- File "requirements.txt":

```
netrecon > requirements.txt
1 flask
2 click
3 asyncio
4 htmx
5 python-dotenv
```

- Tiến hành cài các gói cần thiết: `pip install -r requirements.txt`
- File ".env" điền thông tin username và mật khẩu đã tạo (mật khẩu ứng dụng):

```
netrecon > .env
1 SMTP_USER=hongphuoc123456@gmail.com
2 SMTP_PASS=jyuy phiw pkbf zmom
3
```

- Tạo file "modules/__init__.py".
- Code cho file "modules/banner_grabber.py":

```

netrecon > modules > banner_grabber.py > ...
1  import socket, logging
2  from datetime import datetime
3
4  logging.basicConfig(filename='netrecon.log', level=logging.INFO)
5
6  def log(msg):
7      now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
8      logging.info(f"[{now}] {msg}")
9
10 def grab_banner(ip, port):
11     try:
12         s = socket.socket()
13         s.settimeout(2)
14         s.connect((ip, port))
15         banner = s.recv(1024).decode().strip()
16         s.close()
17         log(f"[{ip}:{port}] Banner: {banner}")
18         return banner
19     except Exception as e:
20         return f"Failed to grab banner: {e}"
21

```

- Code cho file "modules/email_sender.py":

```

netrecon > modules > email_sender.py > ...
1  import smtplib
2  from email.message import EmailMessage
3  from dotenv import load_dotenv
4
5  load_dotenv()
6
7  def send_email(receiver_email, subject, body, smtp_user, smtp_pass):
8      msg = EmailMessage()
9      msg['Subject'] = subject
10     msg['From'] = smtp_user
11     msg['To'] = receiver_email
12     msg.set_content(body)
13
14     try:
15         with smtplib.SMTP_SSL('smtp.gmail.com', 465) as smtp:
16             smtp.login(smtp_user, smtp_pass)
17             smtp.send_message(msg)
18             print(f"[+] Email sent to {receiver_email}")
19     except Exception as e:
20         print(f"[-] Email failed: {e}")
21

```

- Code cho file "modules/filter_utils.py":

```
netrecon > modules > filter_utils.py > ...
1  def filter_targets(ip_list, whitelist=None, blacklist=None):
2      whitelist = set(whitelist or [])
3      blacklist = set(blacklist or [])
4      result = []
5      for ip in ip_list:
6          if blacklist and ip in blacklist:
7              continue
8          if whitelist and ip not in whitelist:
9              continue
10         result.append(ip)
11     return result
12
```

- Code cho file "modules/network_mapper.py":

```
netrecon > modules > network_mapper.py > ...
1  import subprocess, logging
2  from datetime import datetime
3
4  logging.basicConfig(filename='netrecon.log', level=logging.INFO)
5
6  def log(msg):
7      now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
8      logging.info(f"[{now}] {msg}")
9
```

```
10 def map_network():
11     log("Mapping network...")
12     try:
13         arp_output = subprocess.check_output(["arp", "-a"]).decode()
14         log(arp_output)
15         return arp_output
16     except Exception as e:
17         return f"Error: {e}"
18
```

- Code cho file "modules/port_scanner.py":

netrecon > modules > port_scanner.py > ...

```
1 import asyncio, logging, socket
2 from datetime import datetime
3
4 logging.basicConfig(filename='netrecon.log', level=logging.INFO)
5
6 def log(message):
7     now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
8     logging.info(f"[{now}] {message}")
9
10 async def scan_port(target, port, semaphore):
11     try:
12         async with semaphore:
13             conn = asyncio.open_connection(target, port)
14             reader, writer = await asyncio.wait_for(conn, timeout=1)
15             log(f"Port {port} is open on {target}")
16             print(f"[+] {port}/tcp open")
17             writer.close()
18             await writer.wait_closed()
19     except:
20         pass
21
22 async def async_scan_ports(target, ports, rate_limit=100):
23     semaphore = asyncio.Semaphore(rate_limit)
24     tasks = [scan_port(target, port, semaphore) for port in ports]
25     await asyncio.gather(*tasks)
26
```

- Code cho file "modules/service_detector.py":

```
netrecon > modules > service_detector.py > ...
1 import subprocess, logging
2 from datetime import datetime
3
4 logging.basicConfig(filename='netrecon.log', level=logging.INFO)
5
6 def log(msg):
7     now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
8     logging.info(f"[{now}] {msg}")
9
10 def detect_service(ip, ports):
11     ports_str = ','.join(str(p) for p in ports)
12     cmd = ["nmap", "-sV", "-p", ports_str, ip]
13     log(f"Running service detection on {ip}:{ports_str}")
14     try:
15         result = subprocess.check_output(cmd).decode()
16         log(result)
17         return result
18     except subprocess.CalledProcessError as e:
19         return f"Error: {e.output.decode()}"
20
```

- Code cho file "modules/vuln_checker.py":

```
netrecon > modules > vuln_checker.py > check_vulns
1 VULN_PORTS = {
2     21: "FTP - CVE-2015-3306, CVE-2001-0261",
3     22: "SSH - CVE-2018-15473",
4     23: "Telnet - CVE-2011-4862",
5     80: "HTTP - CVE-2021-41773",
6     443: "HTTPS - CVE-2021-3449"
7 }
8
9 def check_vulns(ports):
10     result = {}
11     for port in ports:
12         if port in VULN_PORTS:
13             result[port] = VULN_PORTS[port]
14     return result
15
```

- Code cho file "templates/index.html":

```
netrecon > templates > <> index.html
1  {% extends 'layout.html' %}
2  {% block content %}
3  <form method="post" action="/scan" hx-post="/scan" hx-target="#results">
4      <label>Target IP:</label>
5      <input type="text" name="target" required><br>
6      <label>Ports (comma-separated):</label>
7      <input type="text" name="ports" value="22,80,443"><br>
8      <label>Mode:</label>
9      <select name="mode">
10         <option value="all">All</option>
11         <option value="scan">Port Scan</option>
12         <option value="service">Service Detection</option>
13         <option value="banner">Banner Grab</option>
14         <option value="map">Network Map</option>
15         <option value="vuln">Vulnerability Check</option>
16     </select><br>
17     <label>Email nhận kết quả:</label>
18     <input type="email" name="email" required><br>
19     <button type="submit">Scan</button>
20 </form>
21
22 <div id="results">
23 </div>
24 {% endblock %}
25
```

- Code cho file "templates/layout.html":

```
netrecon > templates > <> layout.html
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>NetRecon</title>
5      <link rel="stylesheet" href="/static/style.css">
6  </head>
7  <body>
8      <div class="container">
9          <h1>NetRecon - Network Reconnaissance Toolkit</h1>
10         {% block content %}{% endblock %}
11     </div>
12 </body>
13 </html>
14
```

- Code cho file "templates/result.html":

```

netrecon > templates > <> result.html
1  {% if result.scan %}
2  <h2>Scan Result:</h2>
3  <pre>{{ result.scan }}</pre>
4  {% endif %}
5
6  {% if result.service %}
7  <h2>Service Detection:</h2>
8  <pre>{{ result.service }}</pre>
9  {% endif %}
10
11 {% if result.banner %}
12 <h2>Banner Grabbing:</h2>
13 <ul>
14     {% for port, banner in result.banner.items() %}
15         <li>Port {{ port }}: {{ banner }}</li>
16     {% endfor %}
17 </ul>
18 {% endif %}
19
20 {% if result.map %}
21 <h2>Network Map:</h2>
22 <pre>{{ result.map }}</pre>
23 {% endif %}
24
25 {% if result.vuln %}
26 <h2>Vulnerability Check:</h2>
27 <ul>
28     {% for port, vuln in result.vuln.items() %}
29         <li>Port {{ port }}: {{ vuln }}</li>
30     {% endfor %}
31 </ul>
32 {% endif %}
33

```

- Code cho file "static/style.css":

```

netrecon > static > # style.css > ...
1  body {
2      font-family: monospace; background: #1e1e1e;
3      color: #eee; padding: 20px;
4  }
5  input, select, button {
6      margin: 5px; padding: 5px;
7  }
8  .container {
9      max-width: 800px; margin: auto;
10 }
11 pre {
12     background: #2e2e2e; padding: 10px;
13     border-left: 4px solid #4CAF50;
14 }
15

```

- Code cho file "cli.py":

netrecon > cli.py > ...

```
1 import click, asyncio
2 from modules.port_scanner import async_scan_ports
3 from modules.service_detector import detect_service
4 from modules.banner_grabber import grab_banner
5 from modules.network_mapper import map_network
6 from modules.vuln_checker import check_vulns
7
8 @click.command()
9 @click.option('--target', prompt='Target IP', help='Target IP address.')
10 @click.option('--ports', default='22,80,443',
11               help='Comma-separated port list or range.')
12 @click.option('--rate-limit', default=100, help='Max concurrent scans.')
13 @click.option('--mode', default='all',
14               help='Choose from: scan, service, banner, map, vuln, all')
15 def cli(target, ports, rate_limit, mode):
16     ports_list = list(map(int, ports.split(',')))
17
18     if mode in ['scan', 'all']:
19         asyncio.run(async_scan_ports(target, ports_list, rate_limit))
20     if mode in ['service', 'all']:
21         print(detect_service(target, ports_list))
22     if mode in ['banner', 'all']:
23         for p in ports_list:
24             print(f"{p}: {grab_banner(target, p)}")
25     if mode in ['map', 'all']:
26         print(map_network())
27     if mode in ['vuln', 'all']:
28         print(check_vulns(ports_list))
29
30 if __name__ == '__main__':
31     cli()
32
```

- Code cho file "app.py":

netrecon > app.py > ...

```
1 from flask import Flask, render_template, request
2 from modules import port_scanner, service_detector, banner_grabber
3 from modules import network_mapper, vuln_checker, email_sender
4 import asyncio, os
5
6 app = Flask(__name__)
7
```

```

8 @app.route('/')
9 def index():
10     return render_template('index.html')
11
12 @app.route('/scan', methods=['POST'])
13 def scan():
14     target = request.form['target']
15     ports = list(map(int, request.form['ports'].split(',')))
16     mode = request.form['mode']
17     email = request.form['email']
18     result = {}
19
20     if mode in ['scan', 'all']:
21         result['scan'] = asyncio.run(port_scanner.async_scan_ports(target,
22                                                                     ports))
23     if mode in ['service', 'all']:
24         result['service'] = service_detector.detect_service(target, ports)
25     if mode in ['banner', 'all']:
26         result['banner'] = {port:
27                             banner_grabber.grab_banner(target, port) for port in ports}
28     if mode in ['map', 'all']:
29         result['map'] = network_mapper.map_network()
30     if mode in ['vuln', 'all']:
31         result['vuln'] = vuln_checker.check_vulns(ports)
32
33     body = "Kết quả NetRecon:\n\n"
34     for k, v in result.items():
35         body += f"--- {k.upper()} ---\n{v}\n\n"
36

```

```

37     smtp_user = os.getenv("SMTP_USER")
38     smtp_pass = os.getenv("SMTP_PASS")
39     email_sender.send_email(email, "Kết quả quét từ NetRecon", body,
40                             smtp_user, smtp_pass)
41
42     return render_template('result.html', result=result)
43
44 if __name__ == '__main__':
45     app.run(debug=True, host='0.0.0.0', port=5000)
46

```

- Thêm vào file ".gitignore":

```

.gitignore
5 gitsecure.log
6 certs/
7 *.pem
8 .env

```

- Chạy kiểm tra "cli.py":

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> cd .\netrecon\
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\netrecon> python .\cli.py
Target IP: 10.14.89.200
Starting Nmap 7.97 ( https://nmap.org ) at 2025-06-12 10:09 +0700
Nmap scan report for 10.14.89.200
Host is up (0.0010s latency).

PORT      STATE SERVICE VERSION
22/tcp    closed ssh
80/tcp    closed http
443/tcp   closed https

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.12 seconds

22: Failed to grab banner: timed out
80: Failed to grab banner: timed out
443: Failed to grab banner: timed out

Interface: 169.254.46.41 --- 0x9
  Internet Address      Physical Address      Type
  169.254.255.255       ff-ff-ff-ff-ff-ff    static
  224.0.0.22            01-00-5e-00-00-16    static
  224.0.0.251           01-00-5e-00-00-fb    static
  224.0.0.252           01-00-5e-00-00-fc    static
  230.0.0.1             01-00-5e-00-00-01    static
  239.255.255.250       01-00-5e-7f-ff-fa    static
  255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 10.14.89.200 --- 0x11
  Internet Address      Physical Address      Type
  10.14.0.254           00-aa-6e-37-de-46    dynamic
  10.14.255.255         ff-ff-ff-ff-ff-ff    static
  224.0.0.22            01-00-5e-00-00-16    static
  224.0.0.251           01-00-5e-00-00-fb    static
  224.0.0.252           01-00-5e-00-00-fc    static
  230.0.0.1             01-00-5e-00-00-01    static
  255.255.255.255       ff-ff-ff-ff-ff-ff    static
```

- Test nhanh "cli.py":

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\netrecon> python cli.py --target scanme.nmap.org --ports 22,80 --mode scan
[+] 80/tcp open
[+] 22/tcp open
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\netrecon> █
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\netrecon> python cli.py --target 192.168.1.1 --ports 21,22,80,443 --mode all
[+] 80/tcp open
Starting Nmap 7.97 ( https://nmap.org ) at 2025-06-12 10:12 +0700
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 8.13 seconds
```

- Kiểm tra "app.py":

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249\netrecon> python .\app.py
* Serving Flask app 'app'
* Debug mode: on
█
```

- Vào `http://localhost:5000/` để kiểm tra ứng dụng. Điền các thông số cần thiết và nhấn "Scan":

NetRecon - Network Reconnaissance Toolkit

Target IP: 10.14.89.200

Ports (comma-separated): 22,80,443

Mode: All

Email nhận kết quả: hongphuoc2597@gmail.com

Scan

- Kết quả phản hồi:

Service Detection:

Starting Nmap 7.97 (<https://nmap.org>) at 2025-06-12 10:15 +0700
Nmap scan report for 10.14.89.200
Host is up (0.0010s latency).

PORT	STATE	SERVICE	VERSION
22/tcp	closed	ssh	
80/tcp	closed	http	
443/tcp	closed	https	

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .
Nmap done: 1 IP address (1 host up) scanned in 1.12 seconds

Banner Grabbing:

- Port 22: Failed to grab banner: timed out
- Port 80: Failed to grab banner: timed out
- Port 443: Failed to grab banner: timed out

Network Map:

Interface: 169.254.46.41 --- 0x9

Internet Address	Physical Address	Type
169.254.255.255	ff-ff-ff-ff-ff-ff	static
224.0.0.22	01-00-5e-00-00-16	static
224.0.0.251	01-00-5e-00-00-fb	static
224.0.0.252	01-00-5e-00-00-fc	static
230.0.0.1	01-00-5e-00-00-01	static
239.255.255.250	01-00-5e-7f-ff-fa	static
255.255.255.255	ff-ff-ff-ff-ff-ff	static

- Kiểm tra email và thấy email thông báo từ hệ thống:

Kết quả quét từ NetRecon Hộp thư đến x



hongphuoc123456@gmail.com

đến tôi ▾

Kết quả NetRecon:

--- SCAN ---

None

--- SERVICE ---

Starting Nmap 7.97 (<https://nmap.org>) at 2025-06-12 10:15 +0700

Nmap scan report for 10.14.89.200

Host is up (0.0010s latency).

PORT	STATE	SERVICE	VERSION
------	-------	---------	---------

22/tcp	closed	ssh	
--------	--------	-----	--

80/tcp	closed	http	
--------	--------	------	--

443/tcp	closed	https	
---------	--------	-------	--

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/>.

Nmap done: 1 IP address (1 host up) scanned in 1.12 seconds

--- BANNER ---

{22: 'Failed to grab banner: timed out', 80: 'Failed to grab banner: timed out', 443: 'Failed to grab banner: timed out'}

--- MAP ---

- Commit code, push code lên Github:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git add .
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git commit -m "[add] netrecon"
GitSecure: All checks passed.
[main 5506b83] [add] netrecon
17 files changed, 511 insertions(+), 1 deletion(-)
create mode 100644 netrecon/app.py
create mode 100644 netrecon/cli.py
create mode 100644 netrecon/modules/__init__.py
create mode 100644 netrecon/modules/banner_grabber.py
create mode 100644 netrecon/modules/email_sender.py
create mode 100644 netrecon/modules/filter_utils.py
create mode 100644 netrecon/modules/network_mapper.py
create mode 100644 netrecon/modules/port_scanner.py
create mode 100644 netrecon/modules/service_detector.py
create mode 100644 netrecon/modules/vuln_checker.py
create mode 100644 netrecon/netrecon.log
create mode 100644 netrecon/requirements.txt
create mode 100644 netrecon/static/style.css
create mode 100644 netrecon/templates/index.html
create mode 100644 netrecon/templates/layout.html
create mode 100644 netrecon/templates/result.html
PS C:\Users\HONGPHUOC\Desktop\TH_LTANTT_1511060249> git push origin main
```